

UNIVERSIDAD NACIONAL DE AVELLANEDA

DPTO. DE TECNOLOGÍA Y ADMINISTRACIÓN

INGENIERÍA EN INFORMÁTICA

Sistemas de Control Automático

Trabajo de Laboratorio 1

Control Encendido-Apagado de una Cinta Transportadora

ESTUDIANTES

D' ERAMO, Francisco A

franciscoderamo@gmail.com

LÓPEZ, Agustín

agustin.j.r@hotmail.com

MORÁN, Iván

ivygreengots@gmail.com

PEDEMONTE, Yoel

yoelpedemonte@gmail.com

DOCENTES

Guillermo Federico Caporaletti

gfcaporaletti@undav.edu.ar

Juan Castellano

jcastellano@undav.edu.ar

1er Cuatrimestre 2023

Índice

Índice **2**
Objetivo **3**
Introducción **4**
Marco Teórico **5**
Arduino Mega 2560 5
Relé 7
Optoacoplador 8
Pulsador 10
Diseño **12**
Desarrollo **15**
Mediciones **18**
Medición 1 18
Medición 2 21
Medición 3 23
Conclusión **27**
Mejoras a desarrollar **29**
Anexo 1: Módulos **30**
Anexo 2: Medición 1 ..... **40**
Anexo 3: Medición 2 ..... **44**
Anexo 4: Medición 3 ..... **48**
Bibliografía **52**
Referencias **53**

Objetivo

El objetivo de este trabajo de laboratorio es controlar un sistema físico de nuestra elección, en nuestro caso una cinta transportadora, mediante una acción de control de encendido-apagado (ON-OFF).

La experiencia debe ser registrada para poder medir la respuesta del sistema a un escalón y evaluar la reacción del sistema ante alguna perturbación. Sería deseable poder encontrar la máxima precisión posible con este modo de acción de control; y es necesario utilizar o reconfigurar la experiencia para medir los parámetros del sistema físico.

Introducción

Para el presente trabajo de laboratorio decidimos en grupo optar por realizar el control encendido-apagado de una cinta transportadora.

La cinta transportadora elegida cuenta con una cinta de unos 35 cm de largo, 6 rodillos de aprox. 1 cm de diámetro, un *motor DC* y un *sensor optoacoplador.*



Figura 1: Fotografía de la cinta transportadora elegida.

Nuestro objetivo será controlar dicho sistema físico de tipo mecánico con una acción de control respecto a un rango de velocidad dado por el usuario (en nuestro caso la velocidad finalmente fue determinada por nosotros mismos) a través de una placa Arduino Mega 2560.

Se buscará la máxima precisión posible con este modo de acción de control. Es por ello que, planteamos realizar variaciones en los parámetros a manipular para reconfigurar la experiencia y realizar diferentes mediciones con la finalidad de analizar y comparar los resultados obtenidos.

Marco Teórico

Arduino Mega 2560

El Arduino Mega 2560 es una placa de microcontrolador basada en el ATmega2560. Tiene 54 pines de entrada/salida digital (de los cuales 15 se pueden usar como salidas PWM), 16 entradas analógicas, 4 UART (puertos seriales de hardware), un oscilador de cristal de 16 MHz, una conexión USB, un conector de alimentación, un cabezal ICSP, y un botón de reinicio. Contiene todo lo necesario para soportar el microcontrolador.

Para comenzar a utilizarlo simplemente se debe conectar a una computadora con un cable USB o encenderlo con un adaptador de CA a CC o una batería.

Especificaciones técnicas:

Microcontroller	ATmega2560
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	54 (of which 15 provide PWM output)
Analog Input Pins	16
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	256 KB of which 8 KB used by bootloader
SRAM	8 KB
EEPROM	4 KB
Clock Speed	16 MHz
LED_BUILTIN	13
Length	101.52 mm
Width	53.3 mm
Weight	37 g

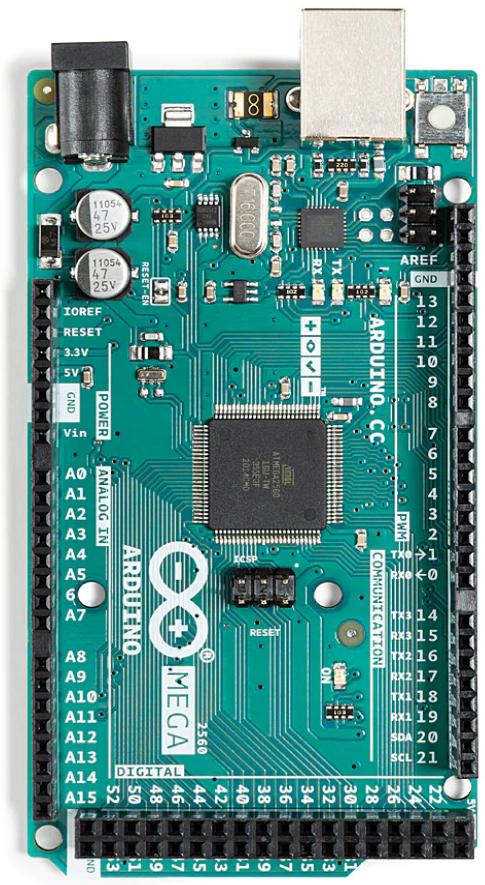


Figura 2: Especificaciones técnicas de Arduino Mega y *Placa de Arduino Mega 2560 Rev3*

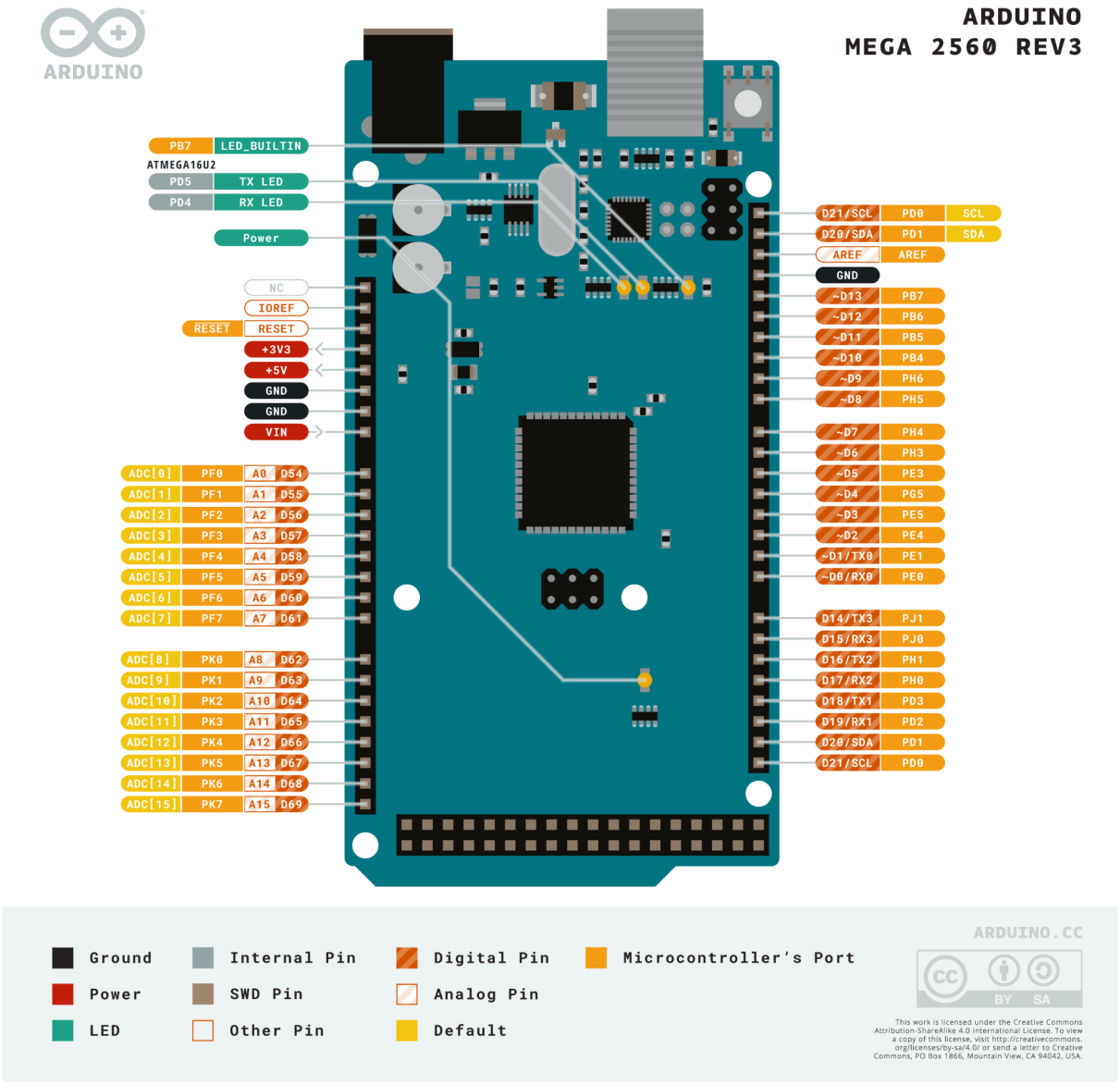
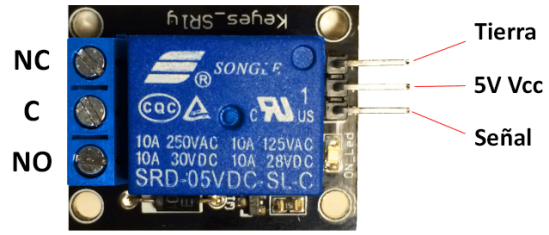


Figura 3: Diagrama de pines del Arduino Mega 2560 Rev3

El entorno de desarrollo integrado de Arduino es una aplicación multiplataforma que está escrita en el lenguaje de programación Java. La aplicación Arduino IDE es de código abierto, su código fuente está alojado en GitHub y su versión actual es la 2.0.

Relé

Un relé es un dispositivo electromagnético que cuando recibe electricidad es capaz de abrir o cerrar una serie de contactos que lleva en su interior. Está formado por una bobina con un núcleo de hierro y uno o más contactos.



NC: Normally Closed (Normalmente Cerrado)
C: Common Terminal (Terminal en Común)
NO: Normally Open (Normalmente Abierto o NA)

Figura 4: Terminales y Pines de un Relay de 5V

Existen dos tipos de contactos en un relé: Normalmente Abierto (NA ó NO) y Normalmente Cerrado (NC).



Figura 5: Modelo de 2 Relés utilizado en la práctica

Cuando llega electricidad a la bobina esta crea un campo magnético haciendo que cambien de posición los contactos del relé: El NA pasa a cerrado y el NC pasa a abierto. En nuestro caso usamos el modelo SRD-05VDC-SL-C con doble relay pero solo utilizamos uno de ellos.

Optoacoplador

Este módulo usa un sensor de acoplamiento óptico de ranura importado, con un ancho de 5 mm. El principio de funcionamiento del acoplador óptico es que el fotoacoplador produce una corriente óptica debido al efecto fotoeléctrico, que se induce a partir de la salida del fotón y realiza la conversión de luz-electricidad. El acoplador óptico (acoplador, OC) también se conoce como sensor fotoeléctrico o fotoacoplamiento para abreviar. El fotoacoplador transmite señales eléctricas utilizando luz como medio. Los acopladores ópticos se componen generalmente de tres partes: la emisión de luz, la recepción de luz y la amplificación de la señal.

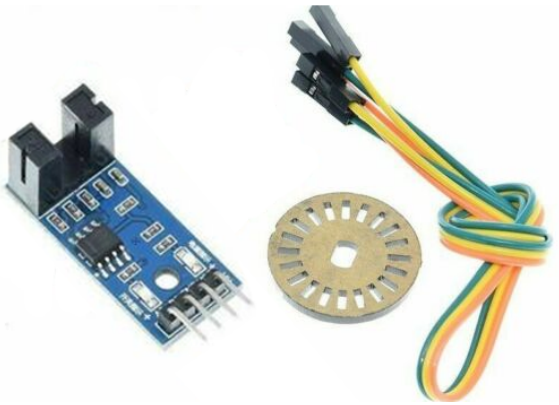


Figura 6: Plaqueta FC-03 - LM393

Con este módulo sensor infrarrojo de velocidad y con el comparador LM393 podemos calcular velocidad. Por ejemplo: la velocidad de rotación de las ruedas de un carro robot, si colocamos una corona dentada que gira unida a una rueda. También se podría usar como un interruptor óptico. El funcionamiento básico de este sensor es el siguiente; Si se hace pasar cualquier cosa entre la ranura del sensor, este crea un pulso digital en el pin D0. Este pulso va de 0V a 5V y es una señal digital TTL. Luego con Arduino podemos leer este pulso. Conocer la posición o velocidad de un motor es muy importante en robótica, para lo cual existen diversas alternativas, siendo una de las más comunes el uso de encoders de tipo óptico. Los encoders en general son dispositivos que se encargan de convertir el movimiento angular o lineal en pulsos eléctricos que puedan ser interpretados por el controlador del sistema. Los encoders incrementales ópticos realizan la medición de movimiento con el uso de un haz de luz infrarrojo que se ve interrumpido por las ranuras de un

disco acoplado al eje. La cantidad de ranuras por vuelta determinará la precisión del encoder.

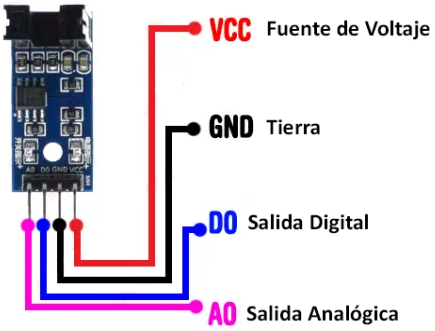


Figura 7: Diagrama del Optoacoplador

Pulsador

Un pulsador electrónico, también conocido como interruptor electrónico o botón electrónico, es un dispositivo utilizado para controlar circuitos electrónicos. A diferencia de los pulsadores mecánicos tradicionales, que utilizan contactos físicos para abrir o cerrar un circuito eléctrico, los pulsadores electrónicos emplean componentes electrónicos para realizar la conmutación.



Figura 8: Botón pulsador grande

Un pulsador electrónico generalmente consta de dos terminales o contactos, que se conectan a los extremos de un circuito. Cuando se presiona el botón o se activa el interruptor, se produce un cambio en la configuración interna del dispositivo, permitiendo que la corriente eléctrica fluya a través de él o se interrumpa. Esto se logra mediante el uso de componentes electrónicos, como transistores, relés, circuitos integrados o microcontroladores, que responden a la señal de entrada generada por la pulsación del botón.

Los pulsadores electrónicos se utilizan en una amplia variedad de aplicaciones, tanto en dispositivos electrónicos de consumo, como teléfonos móviles, computadoras, electrodomésticos y equipos de audio, como en sistemas de control industrial y automotriz. También se emplean en la construcción de paneles de control, tableros de instrumentos y circuitos de automatización, donde proporcionan una forma conveniente de iniciar o detener un proceso, activar una función o cambiar una configuración.

El modo de configuración de los pulsadores se determina con la resistencia conocida como *pull-down* o *pull-up*. Como funcionalidad básica, estas resistencias establecen un estado lógico en un pin o entrada de un circuito lógico cuando se encuentra en estado de reposo. Como bien indica su nombre la resistencia pull up

establece un estado HIGH y las resistencias pull down establecen un estado LOW cuando el pin se encuentra en reposo. Esto evita los falsos estados que se producen por el ruido generado por los circuitos electrónicos.

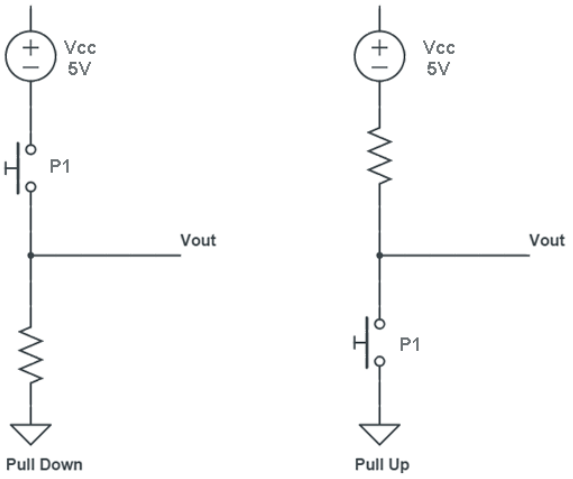


Figura 9: Configuración de resistencia en Pull Down o Pull Up

Normalmente las resistencias que se utilizan en estos casos son de 1K o 10K. Estas dos configuraciones nos evitarán que en estado de reposo midamos un valor erróneo eliminando la influencia de factores externos sobre nuestras mediciones como el ruido eléctrico. Vale aclarar también que en Arduino podemos aplicar una resistencia pull up en las entradas digitales a través de código. Simplemente poniendo un pin en modo INPUT_PULLUP con la instrucción pinMode.

Diseño

El sistema que vamos a controlar se puede representar en un diagrama de bloques, como se observa en la Figura 10. En nuestro caso, la planta es la cinta transportadora, que contiene un motor de DC y un sensor optoacoplador; la *variable controlada* es la velocidad del motor de la cinta; y la *variable manipulada* es el voltaje que recibe el motor de la cinta. Es importante recalcar que para el mismo usamos un controlador ON-OFF (encendido-apagado), tal como se indica en los objetivos.

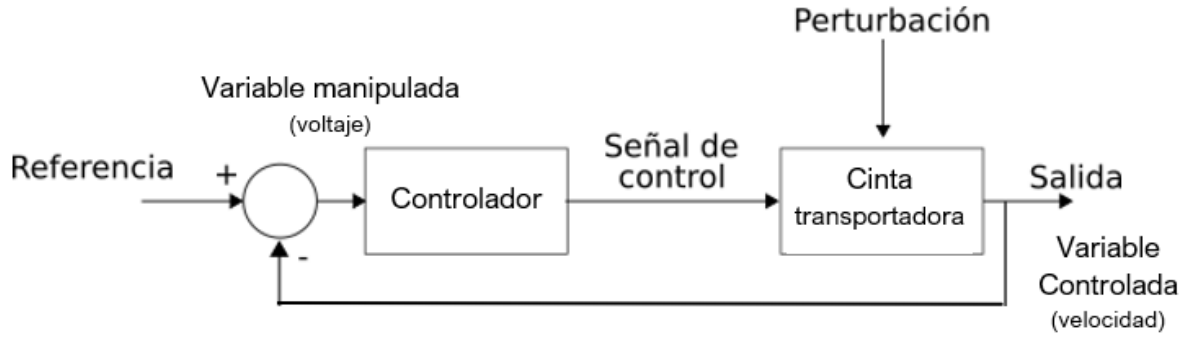


Figura 10: Diagrama de bloques del sistema mecánico elegido.

Rápidamente, al ser un sistema de control ON-OFF, nos damos cuenta que el sistema es no lineal, como todos los sistemas de control ON-OFF. Esto es así ya que el sistema no cumple con el principio de superposición, que establece que la salida de la suma de dos entradas tiene que ser igual a la suma de las salidas de cada entrada por separado.

Además, también podemos darnos cuenta que nuestro sistema de control es invariante en el tiempo, ya que, aunque posee parámetros o componentes que son afectados por uso, corrosión o envejecimiento del material (como es el caso de la cinta en sí misma), no consideramos que son suficientemente importantes para exigir ser considerados en nuestro diseño.

Finalmente, nuestro sistema de control funciona en tiempo discreto, ya que las señales en el sistema están en la forma de pulsos o de un código digital. De esta manera, conocemos la señal de error en cada muestreo pero no recibimos información acerca del error durante los periodos entre dos muestras consecutivas. Sin embargo, al ajustar el tiempo entre muestras de manera que sea lo suficientemente chico (adecuado a nuestro sistema), a los efectos del diseño

podemos considerar como que nuestro sistema es un sistema de control en tiempo continuo.

Ahora bien, previo a comenzar con el desarrollo de nuestro sistema de control, también realizamos una serie de mediciones sobre nuestro sistema, funcionando con un voltaje de 6V. Estas mediciones arrojaron que nuestro motor realiza aproximadamente 4 vueltas por segundo; que consume alrededor de 100 mA sin carga; y que consume alrededor de 550 mA con carga. Como nuestro motor tiene 20 ranuras, que pueden realizar dos cambios de estado, entonces vamos a tener 40 cambios de estado por vuelta. Por lo tanto, estimamos que nuestro motor tendrá 160 cambios de estado por segundo, es decir una frecuencia de 160 Hz, equivalente a 6,25 ms. Esta medición inicial de cuatro giros por segundo además nos permite determinar que nuestro motor hará aproximadamente 240 vueltas por minuto o, de forma equivalente, que nuestro motor tendrá una velocidad máxima de 240 RPM.

Adicionalmente realizamos otras mediciones de menor voltaje para observar cómo variaba nuestro sistema a partir de los distintos niveles de voltaje. Por ejemplo, a 5V medimos que el motor realiza un poco menos de cuatro vueltas en un segundo, podríamos decir 3,5 vueltas por segundo, es decir una velocidad del motor de 210 RPM; a 4V medimos que realizó aproximadamente 3 vueltas por segundo, es decir una velocidad del motor de 180 RPM; a 3V medimos que realizó aproximadamente 2 vueltas por segundo, es decir una velocidad del motor de 120 RPM; y a 1,7V medimos que realizó aproximadamente 1 vuelta por segundo, es decir una velocidad del motor de 60 RPM. De esta manera, como se puede ver en la Figura 11, nos damos cuenta que al aumentar el voltaje, la velocidad aumenta también. Por otro lado, probamos el funcionamiento de la cinta transportadora con y sin carga, y observamos que a medida que se incrementa el peso de la carga su velocidad disminuye. A partir de estas mediciones, determinamos que el rango de valores deseados/objetivo para nuestra variable manipulada estará entre 3V y 6V, para poder mantener nuestra velocidad del motor por encima de 100 RPM aproximadamente.

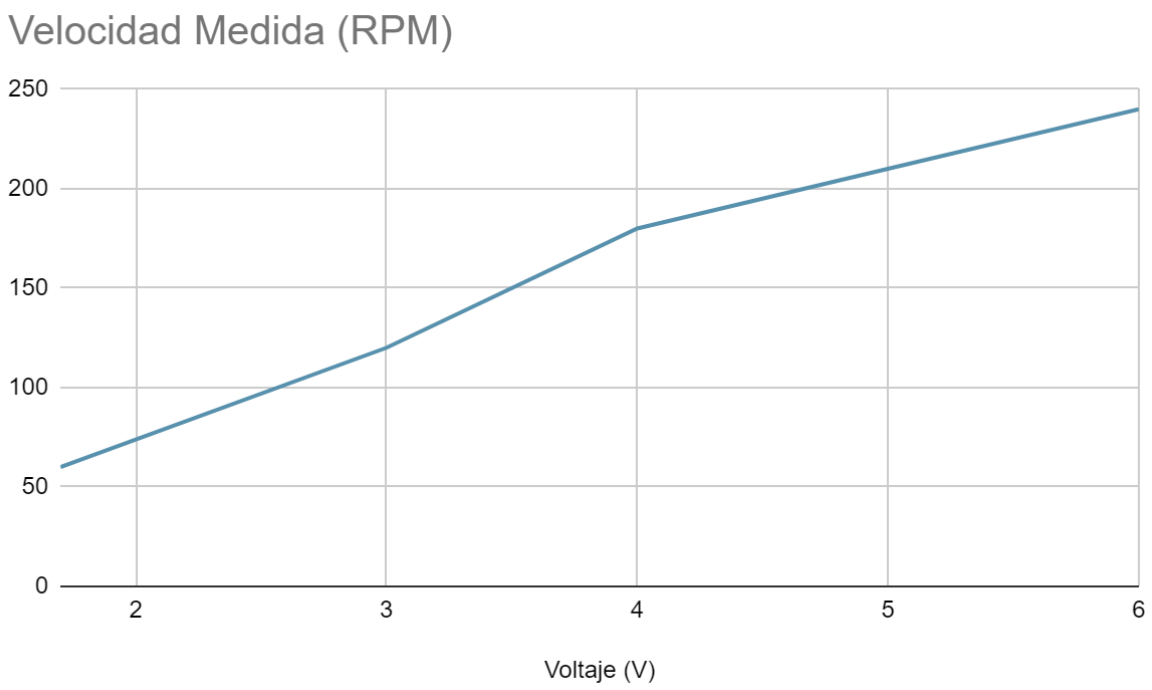


Figura 11: Mediciones de Velocidad RPM dependiendo del Voltaje Aplicado.

Desarrollo

Para poder llevar adelante el trabajo de laboratorio, lo primero que tuvimos que hacer fue agregar la cinta a nuestro sistema físico, ya que no contaba con la misma. Posteriormente investigamos cómo funcionan los componentes que íbamos a utilizar y analizamos el funcionamiento de nuestro sistema.

En un comienzo intentamos utilizar un Shield Motor v1 Driver L293D que fue diseñado por Adafruit para facilitar el control de motores con Arduino. El shield facilita el control de hasta 4 motores DC.[1] Sin embargo, nos encontramos con el problema de que este usaba una librería muy compleja, la cual ya contaba con métodos de control y abstraía de manera difícil de rastrear la definición de los pines de conexión del motor detrás de una placa integrada.

Independientemente de esto último, intentamos utilizarlo en una primera instancia pero, luego de un tiempo y tras varias pruebas, notamos que el conexionado del Shield no estaba funcionando como era esperado. Es por ello que, terminamos desistiendo de usarlo y, en vez, decidimos usar un relé (relay) modelo SRD-05VDC-SL-C. Este tipo de relé tiene doble relay pero en nuestro caso utilizamos uno solo.

Procedimos al conexionado de los diferentes componentes al Arduino. En una primera instancia probamos conectar el “botón de emergencia” en pull-down utilizando una resistencia de 1K conectados a un protoboard y alimentando el Arduino a través de los 5V del USB de la notebook. La funcionalidad de este botón de emergencia es que, cuando se presiona, el sistema debe parar inmediatamente y prender un led de emergencia (usamos el led integrado del Arduino) que titile; acto seguido, no se debe dejar que el sistema arranque de nuevo hasta que se reinicie.

Una vez que testeamos que el botón funciona correctamente procedimos al conexionado del sensor optoacoplador FC-03-LM393, conectando sus pines VCC y GND al protoboard y utilizando su salida digital D0 al Arduino Mega.

Luego utilizamos una fuente de alimentación regulable para controlar el voltaje y limitar la corriente. Es decir, conectamos el positivo y negativo de la fuente con 6V al relé a través de NC (Normally Closed) y del C (Common Terminal). Del

[1] https://naylampmechatronics.com/ardusystem-shields/86-shield-motor-v1-driver-l293d.html

relé conectamos a los pines de 5V y Tierra al VCC y GND del Arduino, mientras que el pin de señal del relé lo conectamos al pin 5 del Arduino.

Gráficamente, nuestro conexionado nos quedó como se puede ver en las Figuras 12 y 13.

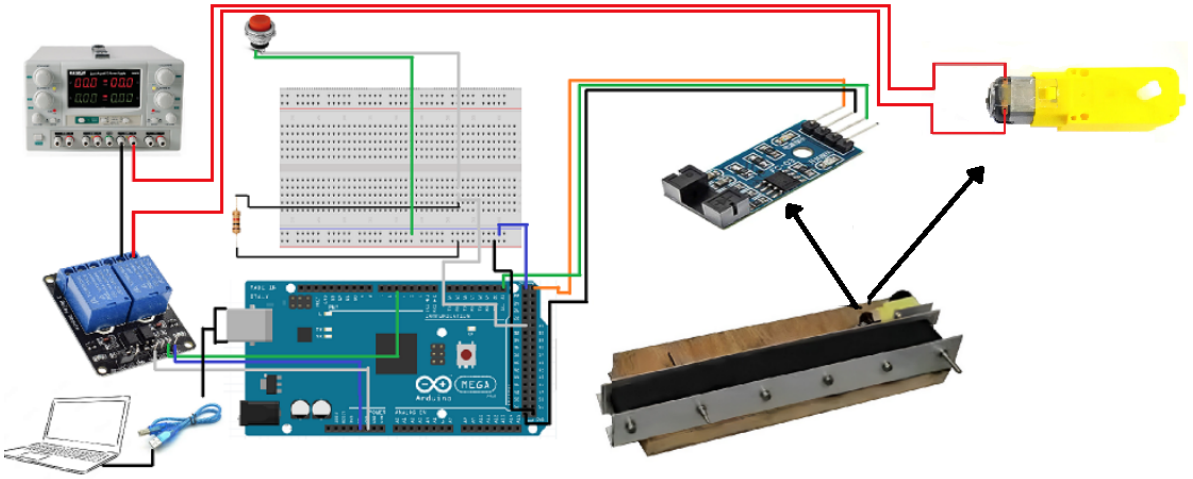


Figura 12: Diagrama del conexionado de nuestro sistema mecánico elegido.

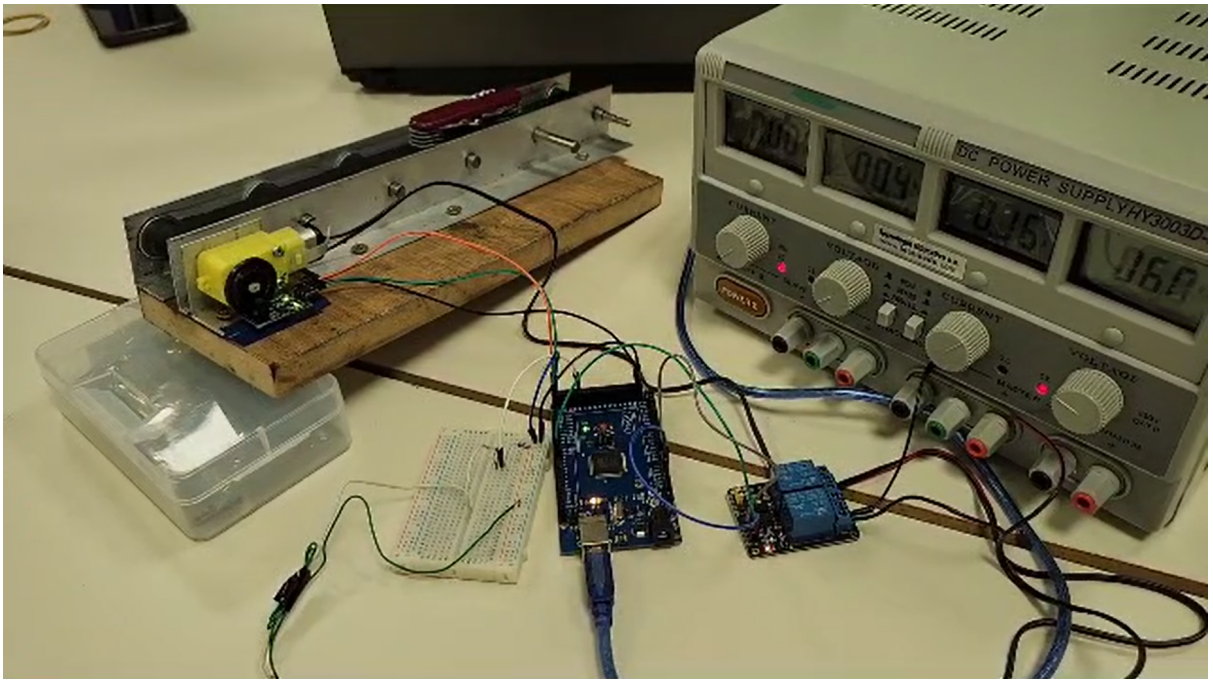


Figura 13: Fotografía del conexionado de la cinta transportadora.

Para el desarrollo de nuestro proyecto, aunque la idea original hubiera sido que el usuario pudiese regular el voltaje objetivo mediante, por ejemplo, un potenciómetro o por pantalla; nos vimos enfrentado a una serie de inconvenientes,

que ya hemos detallado, lo cual nos forzó a tener que hardcodear una velocidad RPM objetivo directamente en el código. A pesar de estos problemas, pudimos realizar el desarrollo del código, el cual puede ser visto en los Anexos, de forma satisfactoria.

En el mismo contamos con un control ON-OFF que, a partir del valor hardcodeado de velocidad RPM objetivo y a partir de un valor hardcodeado de margen de error, nos permite definir la ventana de histéresis de nuestro sistema. También contamos con un módulo de velocidad que, mediante el sensor optoacoplador, nos permite medir la velocidad RPM de nuestro motor. Luego de los seteos iniciales, nuestro programa chequea inicialmente si se accionó el botón de emergencia para parar el sistema y luego, cada cierto período de tiempo de muestreo, nuestro sistema va a leer, controlar y mostrar. Es decir, va a realizar la/s medición/es que correspondan (en nuestro caso, de velocidad); va a controlar usando el controlador ON-OFF, lo cual hará que se prenda o apague el motor dependiendo de un valor booleano de estado; y va a imprimir las estadísticas que nos interesan por pantalla.

Mediciones

Ya habiendo hecho el código y las conexiones de nuestro sistema, podemos pasar a la experiencia en sí. Lo que hicimos fue realizar tres mediciones distintas a partir de definir, para cada medición, un objetivo de velocidad RPM y un márgen de error para el mismo.

Medición 1

En nuestra primera medición, que realizamos cada 10 ms, establecimos un objetivo de 150 RPM con un margen de error de ±20 RPM. Es decir que la ventana de histéresis de nuestro sistema estará entre 130 RPM y 170 RPM. Podemos ver que inicialmente, con una velocidad medida de 0 RPM y un estado = 0 que significa que el motor está apagado, rápidamente nuestro controlador establecerá el estado = 1 para prenderlo. Nuestro motor entonces arrancará y su velocidad rápidamente se disparará a valores que están por encima de los 100 RPM, como 129 RPM o 197 RPM. Al llegar a este último valor, sin embargo, nos habremos pasado de los márgenes establecidos para la ventana de histéresis del control ON-OFF. De esta manera, el valor del estado cambia a estado = 0 y el motor comienza a detenerse al no recibir más voltaje. Aquí entonces, la velocidad comenzará a bajar progresivamente (se obtuvieron, sin embargo, una serie de errores en la medición mediante el cual la velocidad sigue subiendo por unas muestras más) hasta volver a quedar dentro del rango de operación establecido. Finalmente, al bajar tanto la velocidad que se sale del límite inferior de la ventana de histéresis, como por ejemplo 123 RPM, el motor vuelve a prenderse y la velocidad vuelve a aumentar, manteniendo el mismo principio de funcionamiento ya descrito.

De esta medición podemos analizar que:

- En el arranque (los primeros 350 ms), el estado = 1 perdura durante muy poco tiempo (tres muestras); y al motor le cuesta más apagar el motor y comenzar a bajar la velocidad la primera vez (veinte muestras). De esta manera, podemos analizar que tenemos un efecto de inercia en el arranque del motor; incluso vemos que a veces las mediciones llegan a 300 RPM, lo cual teóricamente es imposible porque la velocidad máxima es 240 RPM.

- Una vez pasamos este período, el sistema comienza a funcionar de manera más estable, alternando el valor de estado cada, aproximadamente, 10-15 muestras. Vemos además que la velocidad se mantiene muy bien entre 130 RPM y 170 RPM, aunque en algunos casos el motor puede superar estos valores por unos 15 RPM aproximadamente (es decir que puede llegar a 115 RPM y 185 RPM respectivamente). De esta manera, tenemos un margen de error de aproximadamente ±15 RPM, por lo que nuestra precisión no es lo suficientemente buena en este aspecto. Por otro lado, durante las mediciones se registró que la velocidad fue equivalente a nuestro objetivo de 150 RPM un total de catorce veces de entre trescientas muestras, lo cual también da cuenta de la escasa precisión (4,66%) de este tipo de control.
- Existe algo de ruido en las mediciones, ya que en distintos puntos en los que la velocidad debería bajar o subir, la medición nos muestra la acción contraria. Por ejemplo, si la velocidad viene bajando durante tres muestras, de repente vuelve a subir en la cuarta muestra cuando en realidad debería seguir bajando.

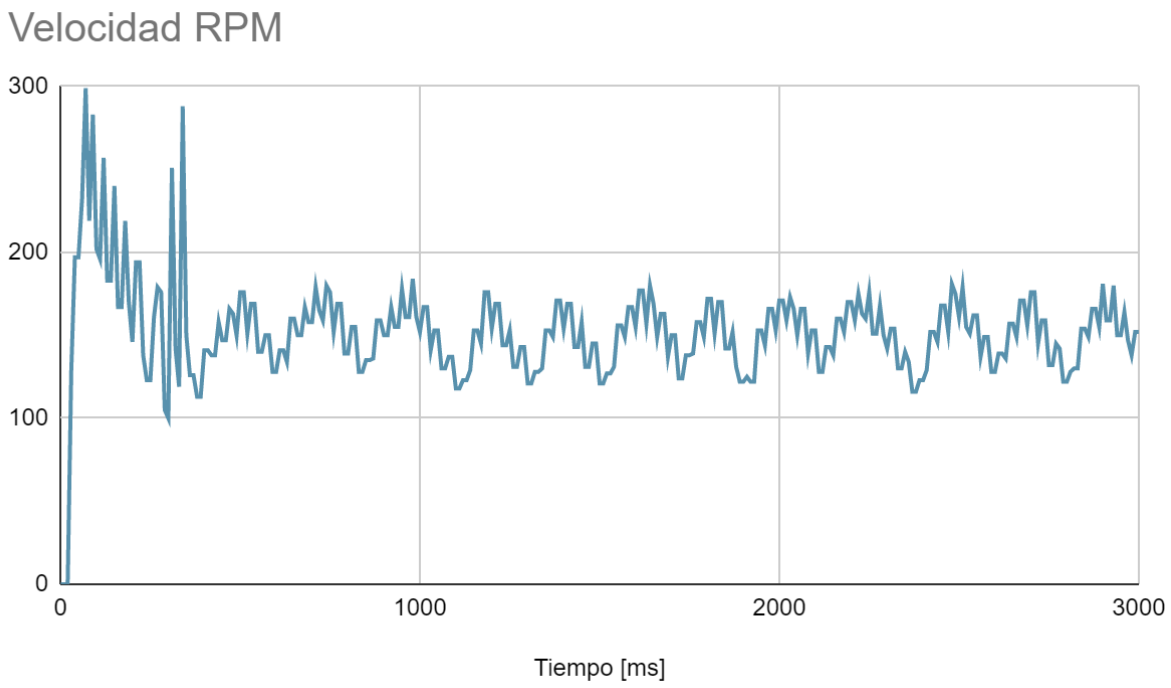


Figura 14: Medición 1 de Velocidad.

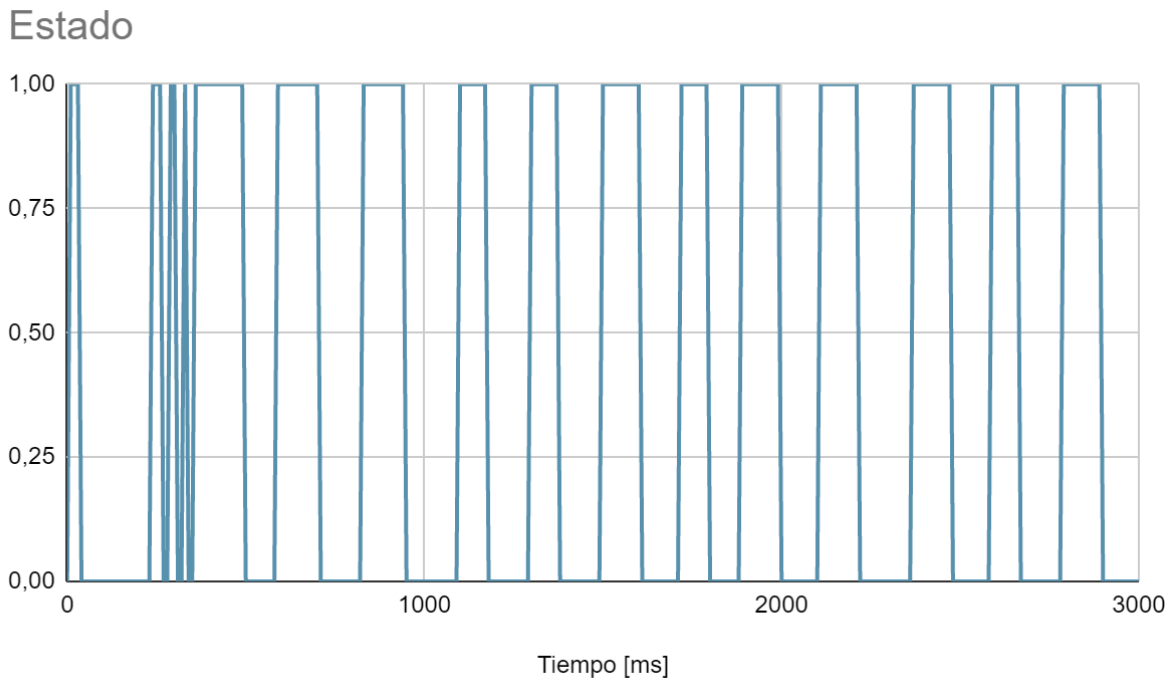


Figura 15: Medición 1 de Estado.

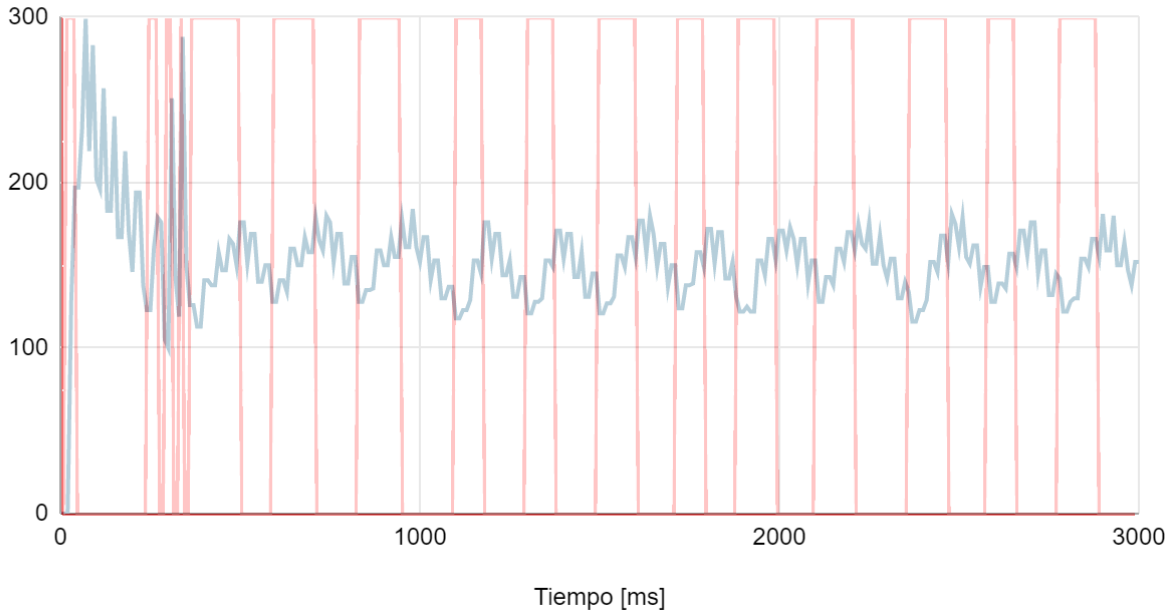


Figura 16: Superposición de la medición 1 de Velocidad y Estado.

Medición 2

Para poder analizar cuánto tarda nuestro sistema en “cargarse”, es decir, en llegar a la velocidad máxima de 240 RPM, realizamos otra medición, ajustando nuestros parámetros para poder tener una ventana de histéresis muy grande. Para esta segunda medición, que realizamos cada 5 ms, establecimos un objetivo de 100 RPM con un margen de error de ±80 RPM. Es decir que la ventana de histéresis de nuestro sistema estará entre 20 RPM y 180 RPM; un rango que fue elegido para poder observar bien qué tanto se dispara nuestra velocidad cuando se enciende el motor (fase de carga) y cómo va disminuyendo la velocidad cuando el mismo se apaga (fase de descarga). Para este segundo caso, observamos un comportamiento similar, como era esperado. Debido a que la medición se realiza cada 5 ms, ahora se tarda aproximadamente cuatro muestras en verse los cambios en la velocidad por parte del ON-OFF; y podemos ver que al prender el motor, en tan sólo 5 ms, es decir en una sola muestra, la velocidad se dispara a valores superiores a los 100 RPM e incluso llegando a los 200 RPM. Sin embargo, al apagarse el motor, la disminución de la velocidad es muchísimo más gradual y toma aproximadamente treinta muestras en quedar por debajo de los 20 RPM.

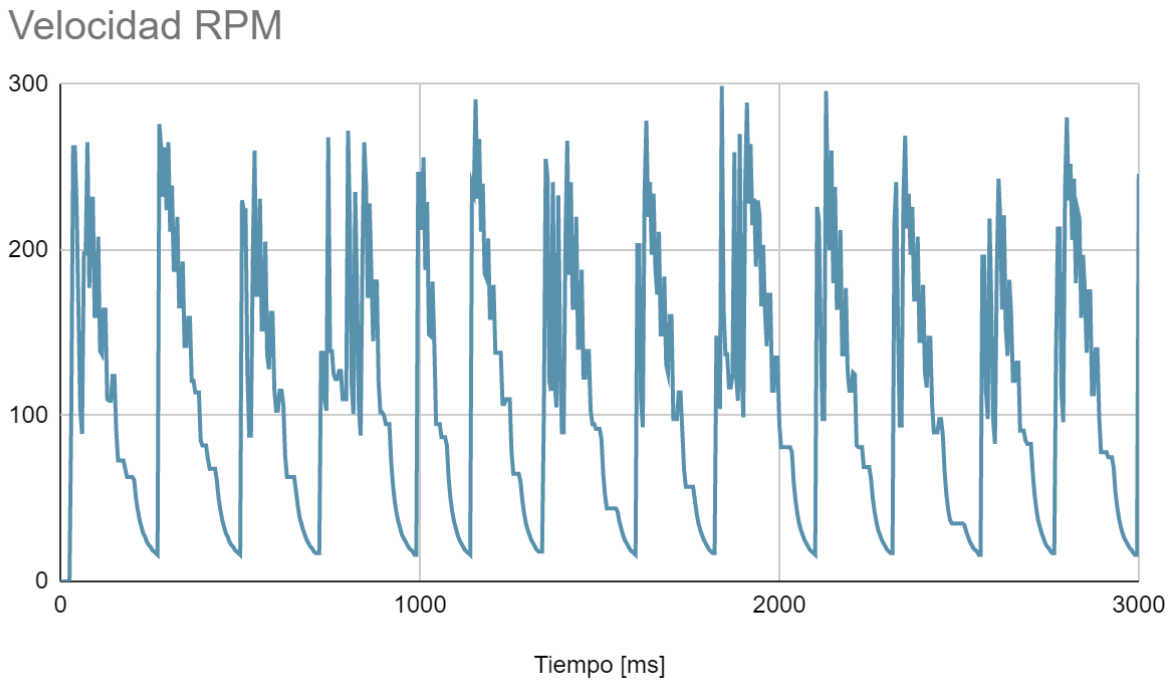


Figura 17: Medición 2 de Velocidad.

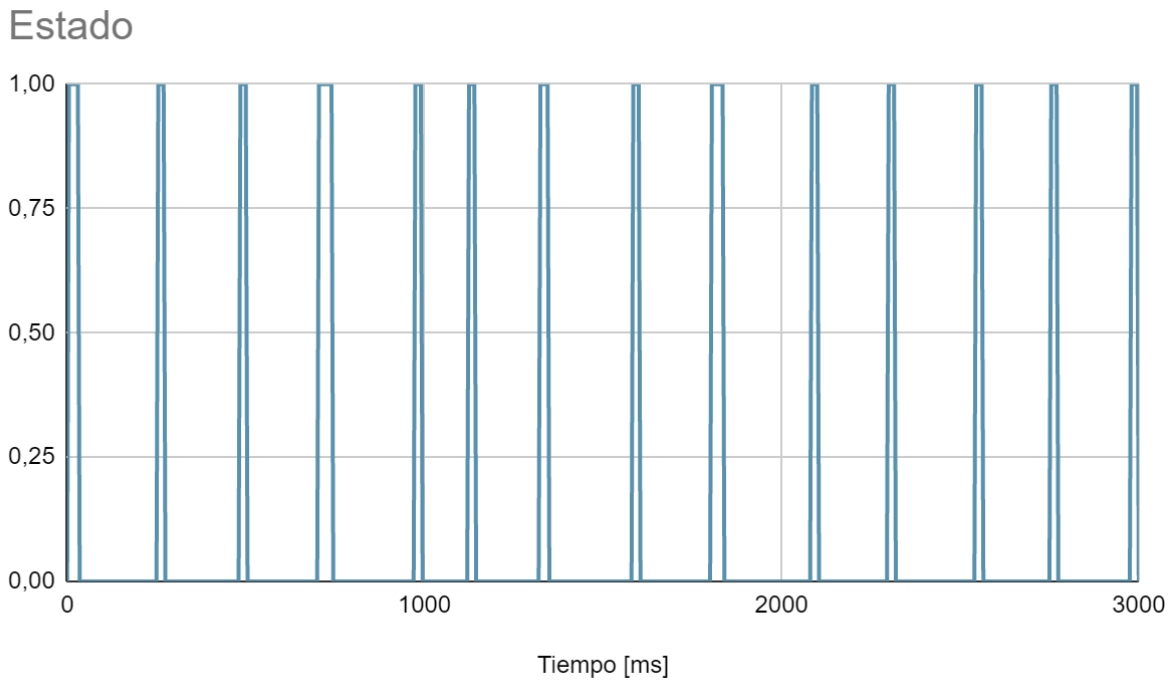


Figura 18: Medición 2 de Estado.

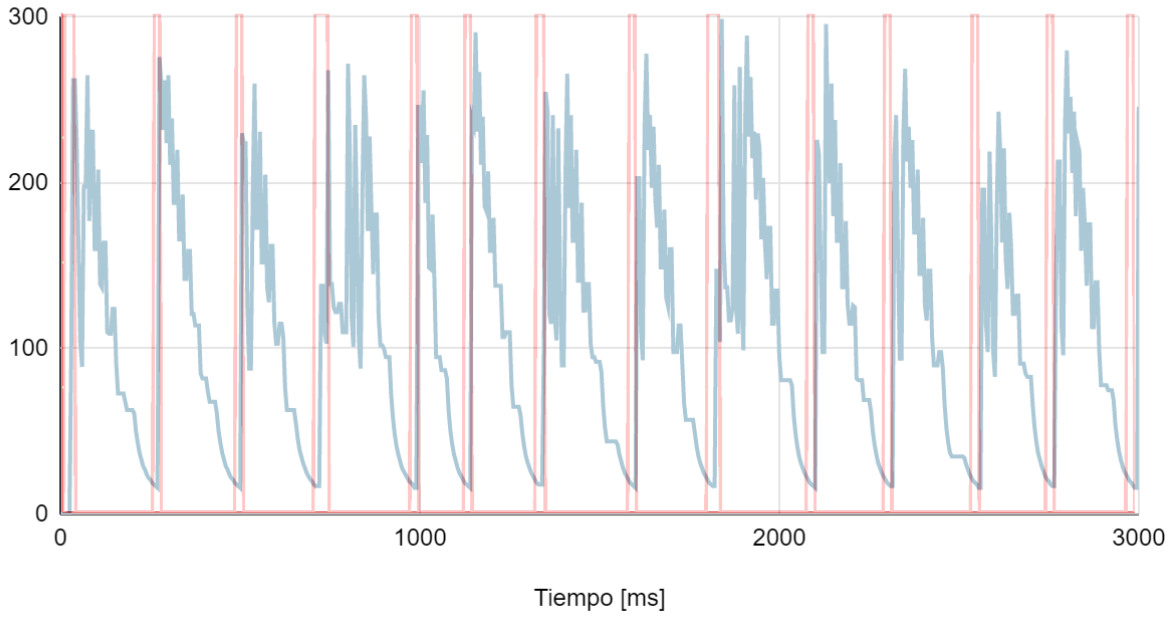


Figura 19: Superposición de la medición 2 de Velocidad y Estado.

A partir de estos datos, graficados en la Figura 17, podemos entonces sacar la constante de tiempo τ de nuestro sistema. En este caso, vamos a sacar el tiempo de descarga, ya que el tiempo de carga generalmente es en tan sólo una muestra de 5 ms. A partir del gráfico, podemos ver que:

- En t = 2610 ms, la velocidad del motor es 243 RPM.
- En t = 2765 ms, la velocidad del motor es 17 RPM.
- Es decir que, en 155 ms, la velocidad bajó unos 226 RPM.

Si tomamos una velocidad inicial de 243 RPM, y consideramos que nuestro sistema es de primer orden y que su función de descarga es la que se puede ver en la Figura 20, entonces podríamos determinar τ a partir de los datos medidos.

$$v(t) = V_i * e^{\frac{-t}{\tau}}$$

Figura 20: Función de fase de descarga de velocidad del motor.

De esta manera, reemplazando con los valores medidos en la ecuación de la Figura 20 y realizando el desarrollo mostrado en la Figura 21, llegamos a la conclusión de que la constante de tiempo τ = 58,29 ms para nuestro sistema (al menos en la fase de descarga).

$$17\ RPM = v(155\ ms) = 243\ RPM * e^{\frac{-155\ ms}{\tau}}$$

$$\frac{17\ RPM}{243\ RPM} \cong 0{,}07 = e^{\frac{-155\ ms}{\tau}}$$

$$\ln(0{,}07) \cong -2{,}659 = -\frac{155\ ms}{\tau}$$

$$\tau \cong \frac{-155\ \text{ms}}{-2{,}659}$$

$$\boxed{\tau \cong 58{,}29\ \text{ms}}$$

Figura 21: Desarrollo para determinar la constante de tiempo τ.

Medición 3

Finalmente, para nuestra tercera medición, que realizamos cada 5 ms, establecimos un objetivo de 130 RPM con un margen de error de ±20 RPM. Es decir

que la ventana de histéresis de nuestro sistema estará entre 110 RPM y 150 RPM. Aquí vemos que el sistema, como habíamos visto antes, tarda aproximadamente una muestra en aumentar su velocidad en 100 RPM (lo cual, si la velocidad ya ha sido reducida por debajo de los 110 RPM, significa que le tomará una sóla muestra de 5 ms para poder elevarse por encima de los 150 RPM que establecimos como límite superior); mientras que tardará entre tres y cinco muestras en reducir su velocidad por debajo de los 110 RPM que establecimos como límite inferior de la ventana de histéresis de nuestro sistema.

Digno de mención de esta medición es que parece que el tiempo de muestreo es mucho más adecuado para el objetivo y la ventana de histéresis que definimos, ya que inmediatamente al cambiar el valor del estado definido por el controlador (de ON a OFF, o de OFF a ON), se puede ver que en la siguiente muestra la velocidad del motor se modifica de manera acorde.

Para esta medición, incluimos además una perturbación en la forma de presionar con nuestro dedo pulgar la cinta, simulando de esta manera la presencia del peso de un objeto. Así, vemos nosotros que se tarda hasta dos muestras en aumentar la velocidad por encima de 150 RPM; y que al reducirse la velocidad cuando se apaga el motor, la misma llega tan bajo como a quedar en 90 RPM (sin peso, nunca se reducía la velocidad por debajo de los 100 RPM). De esta manera, vemos que la velocidad se mantuvo bastante bien dentro de los márgenes deseados, por lo que el sistema respondió satisfactoriamente ante la perturbación.

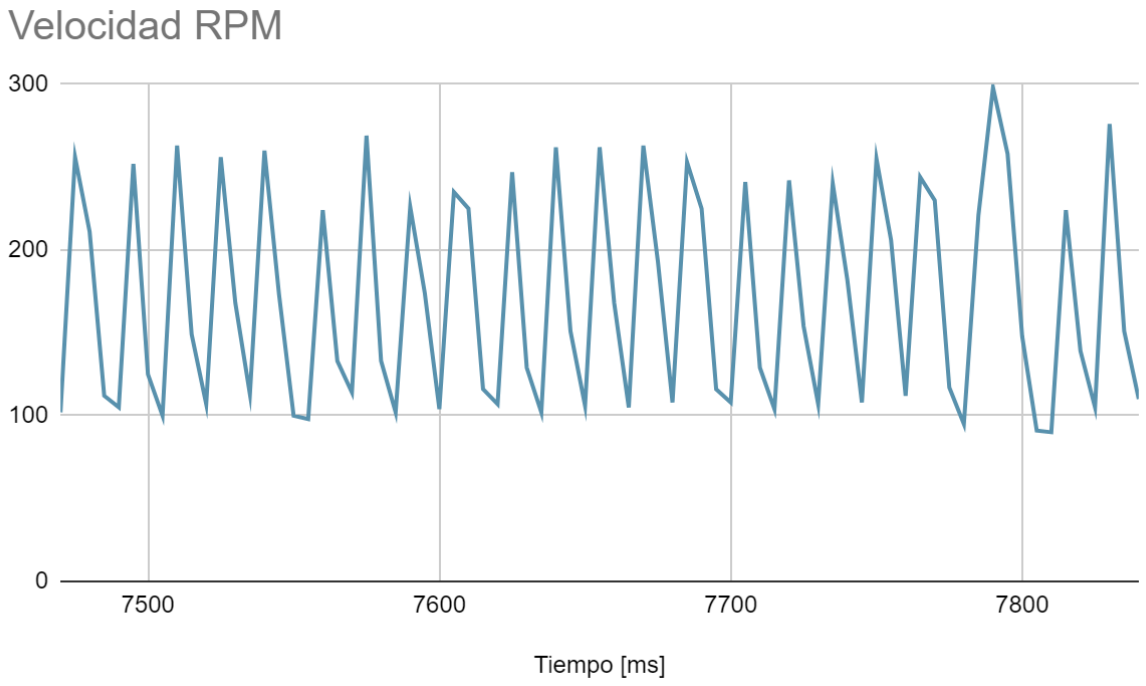


Figura 22: Medición 3 de Velocidad - Perturbación con Peso.

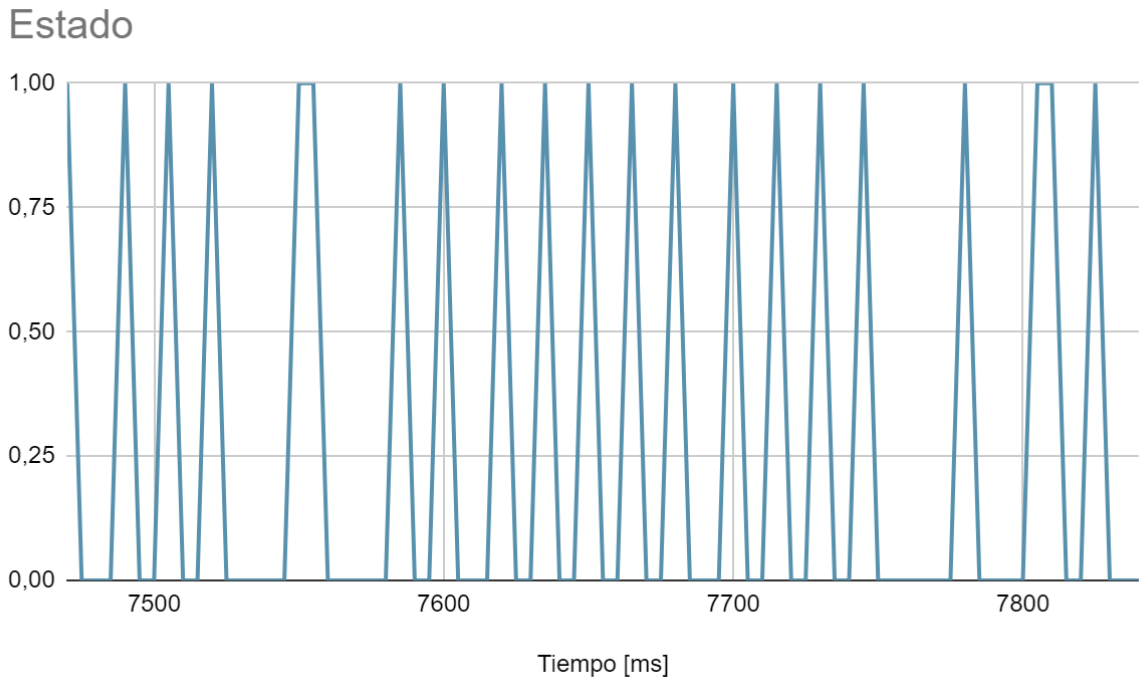


Figura 23: Medición 3 de Estado - Perturbación con Peso.

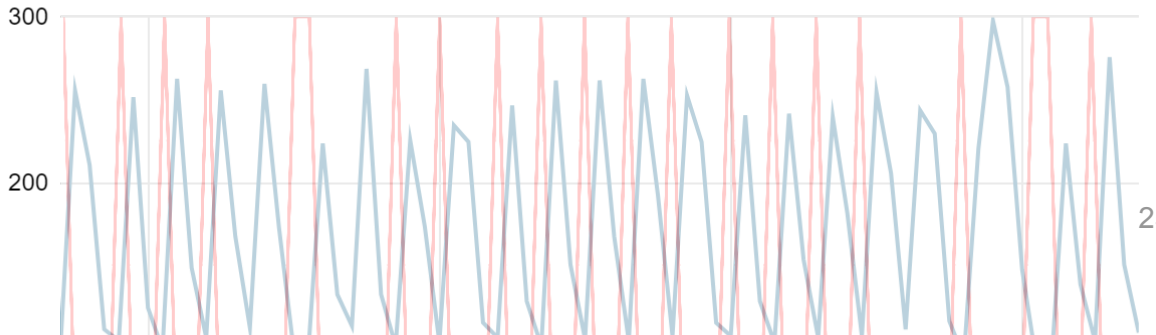


Figura 24: Superposición de las medición 3 de Velocidad y Estado.
Referencias: La línea azul corresponde a la velocidad de la cinta (0 a 300 RPM) y la línea roja corresponde al estado del actuador.

Conclusión

Para iniciar, nos gustaría mencionar primero que logramos concluir el trabajo de laboratorio con éxito haciendo que nuestro control de encendido-apagado funcione correctamente de acuerdo a la velocidad medida y el objetivo determinado. Si bien la precisión lograda no es la mejor, teniendo un margen de error de aproximadamente ±15 RPM para que los valores estén dentro de nuestra ventana de histéresis y alcanzando una precisión del 4,66% para llegar a nuestra velocidad objetivo, sí se puede observar de forma correcta las fluctuaciones que ocurren con la velocidad de nuestro motor mientras va variando para permanecer dentro de la ventana de histéresis que definimos. De todas formas, concluimos que se podría mejorar la precisión de nuestro método de control, aunque la baja precisión es una característica del mismo, por lo que realmente no valdría la pena; sería mucho mejor directamente usar otro tipo de control.

A partir de los gráficos, también identificamos que existe un cierto retardo entre la acción de control y la modificación de la velocidad, pudiendo tardar entre 10 ms y 25 ms para que se vea el efecto de nuestro actuador. Este fenómeno se manifiesta particularmente durante el arranque del motor, es decir, cuando se debe aumentar la velocidad y el estado cambia a un valor igual a 1. Nosotros concluimos que esto se debe parcialmente a un efecto de inercia en el arranque del motor; y a que el tiempo de muestreo empleado no es el adecuado para el tiempo que tarda nuestro actuador en llevar a cabo la acción de control. Durante la tercera medición, sin embargo, el sistema pareciera funcionar de una manera mucho más eficiente al reducir el tiempo de muestreo.

Si bien ya habíamos analizado que nuestro sistema era ~~no~~ físico lineal, en los gráficos se puede ver claramente que contamos con un tiempo de crecimiento que es ampliamente inferior al de decrecimiento, es decir, que es muchísimo más rápido. Esto nos indica una clara alinealidad dentro de nuestro sistema, y es la principal razón por la que no pudimos modelizarlo adecuadamente como un sistema de primer orden.

Concluimos también que algunas perturbaciones que podrían afectar este sistema son cambios que pueden ser desde fluctuaciones en la energía, introducción de distintas cargas, hasta el rozamiento de la propia cinta

transportadora (posible mejora a realizar), o entre otros ejemplos. Para la perturbación que nosotros analizamos, que es la introducción de una carga, el sistema parece responder de forma satisfactoria; sin embargo, hay bastante ruido en todas las mediciones que llevamos a cabo.

A pesar de la aparente buena respuesta ante las perturbaciones y del diseño sencillo que permite que nuestro sistema sea estable, nuestra opinión es que no recomendamos usar un control de encendido-apagado para el control de velocidad de una cinta transportadora, ya que el mismo tiene una baja precisión que es indeseable para lo que se quiere controlar. Si quisiéramos aumentar su precisión, tendríamos que reducir la ventana de histéresis, lo cual aumentaría la cantidad de conmutaciones de encendido y apagado por minuto y reduciría la vida útil del componente. Además, este tipo de control es menos eficiente desde el punto de vista energético, ya que el momento de encendido de un motor no sólo exige a los componentes en general, sino que requiere de una corriente dos o tres veces más grande que su corriente normal de operación, lo cual provoca que la instalación eléctrica deba sobredimensionarse y que haya más pérdidas por este exceso.

Vale aclarar que también tuvimos altibajos mientras realizamos esta experiencia, ya que nos vimos afectados por varios inconvenientes que surgieron en la práctica. Entre los inconvenientes más tediosos podemos destacar la utilización de un pin del Arduino para el sensor que no permite interrupciones, lo cual nos ocasionó una gran pérdida de tiempo hasta lograr encontrar la causa del problema. Otro importante inconveniente fue que al final no pudimos usar el Shield Motor v1 Driver L293D, lo cual nos forzó a realizar numerosas modificaciones a nivel de código que, a su vez, retrasaron el desarrollo de nuestro proyecto.

Sin embargo, a nivel general fue una experiencia muy enriquecedora y positiva. No solo significó el primer proyecto de control automático que realizó cada uno, sino que también fue un desafío tener que enfrentarse a diferentes inconvenientes que fueron surgiendo con el desarrollo del mismo. Por suerte pudimos prevalecer y logramos realizar un trabajo de laboratorio con el cual todos quedamos muy satisfechos.

Mejoras a desarrollar

Como posibles mejoras que podríamos plantear en base a la experiencia que tuvimos, entre muchas otras, sería realizar modificaciones a nivel código para promediar las últimas 2 lecturas con la finalidad de que disminuyan aquellos errores de mediciones en la cual cada ciertas cantidad de lecturas nos arrojaba un valor muy alto o muy bajo respecto a la progresión de las mediciones inmediatas y que no corresponden a un cambio de estado encendido-apagado.

A nivel físico, se podría completar la cantidad de arandelas que separan los rodillos de los laterales y ampliar la cinta de tela realizada al ancho de los rodillos, ambos con la finalidad de reducir las perturbaciones que generan en el sistema en cuestión. Por otro lado, emplear siempre una misma pendiente de manera fija con el objetivo de poder extrapolar las diferentes mediciones realizadas y compararlas efectivamente.

A nivel de diseño, también sería deseable modificar nuestro sistema para que el usuario pueda elegir el voltaje objetivo entre 3V y 6V mediante, por ejemplo, un potenciómetro.

Anexo 1: Módulos

A continuación se coloca el código fuente elaborado para los módulos de velocidad y del control automático de encendido-apagado que se usaron para la cinta transportadora.

controlSiNo.h

```
/*************************************************************************
*************
* Módulo para control SI-NO
*************************************************************************
*************
* Archivo:  controlSiNo.h
* Resumen:  Librería de control si-no elaborada con programación orientada a
objetos (POO)
*           para el curso de Sistemas de Control Automático, UNDAV.
* Autor:    Guillermo Caporaletti <gfcaporaletti@undav.edu.ar>
* Fecha:    Abril 2023
* Version:  1.1
* Cambios:  Incluye la posibilidad de configurar el pin de salida en el
constructor.
*           Agrega la función ConfigurarHisteresis().
*           Versión anterior: 1.0.
*
*************************************************************************
************/

#ifndef CONTROL_SINO_H
#define CONTROL_SINO_H
#include "Arduino.h"

/*************************************************************************
************/

class SiNo                          // Objeto para control SI-NO.
{
  private:
    boolean EstadoSalida;           // Estado de salida del control SI-NO.
    float Prende;                   // Nuestro objetivo es que la ENTRADA
esté entre Prende y Apaga.
    float Apaga;
    int Pendiente;                  // +1 si (APAGA>PRENDE): quiere decir
que el actuador hará que SUBA la ENTRADA
                                    // -1 si (APAGA<PRENDE): quiere decir
```

```
que el actuador hará que BAJE la ENTRADA

  public:
    SiNo(float PRENDE, float APAGA);  // Constructor. Configura los
parámetros PRENDE y APAGA del control si-no.
                                      // Establece por defecto Esdado=false.
    boolean Controlar(float ENTRADA); // Devuelve el estado del controlador
luego de procesar la ENTRADA.
    void Apagar();                    // Pone Estado en false.
    void Prender();                   // Pone Estado en true.
    boolean Estado();                 // Devuelve el Estado de salida.
    void ConfigurarHisteresis(float PRENDE, float APAGA);
                                      // Configura los límites de la
histéresis.
                                      // Es posible cambiar esta
configuración durante la ejecución.
};

/*************************************************************************
***********/

#endif
```

controlSiNo.cpp

```
/*****************************************************************************
*************
* Módulo para control SI-NO
*****************************************************************************
*************
* Archivo:  controlSiNo.cpp
* Resumen:  Libreria de control si-no. Más detalles en "controlSiNo.h".
* Autor:    Guillermo Caporaletti <gfcaporaletti@undav.edu.ar>
* Fecha:    Abril 2023
* Version:  1.1
*
*****************************************************************************
************/

#include "Arduino.h"
#include "controlSiNo.h"

/*****************************************************************************
*************
* Funciones públicas
*****************************************************************************
************/

/*****************************************************************************
*************
* Función:  Constructor SiNo(PRENDE, APAGA)
* Resumen:  Configura los parámetros PRENDE y APAGA del control si-no.
*           Establece por defecto Esdado=false y que no hay pin de salida.
*****************************************************************************
************/
SiNo::SiNo(float PRENDE, float APAGA)
{
  EstadoSalida = false;   // Estado por defecto al iniciar.
  ConfigurarHisteresis( PRENDE, APAGA );
}

/*****************************************************************************
*************
* Función:  Controlar(ENTRADA)
* Resumen:  Devuelve el estado del controlador luego de procesar la ENTRADA.
*           Si fue seteado un PIN de salida, cambia su estado.
*****************************************************************************
************/
boolean SiNo::Controlar(float ENTRADA)       // Controla, teniendo en cuenta
ENTRADA y Estado previo.
{
  if (((ENTRADA-Prende)*Pendiente)<0) {
```

```
    // Debo prender...
    EstadoSalida = true;
  }
  if (((ENTRADA-Apaga)*Pendiente)>0) {
    // Debo apagar...
    EstadoSalida = false;
  }
  return EstadoSalida;
}

/*************************************************************************
*************
* Función:  Apagar()
* Resumen:  Pone en "false" el estado y apaga la salida PIN (si fue
configurada).
*************************************************************************
*************/
void SiNo::Apagar()
{
  EstadoSalida = false;
}

/*************************************************************************
*************
* Función:  Prender()
* Resumen:  Pone en "true" el estado y prende la salida PIN (si fue
configurada).
*************************************************************************
*************/
void SiNo::Prender()
{
  EstadoSalida = true;
}

/*************************************************************************
*************
* Función:  Estado()
* Resumen:  Devuelve el estado del controlador si-no.
*************************************************************************
*************/
boolean SiNo::Estado()
{
  return EstadoSalida;
}

/*************************************************************************
*************
* Función:  ConfigurarHisteresis
```

```
* Resumen:  Establece los umbrales de la histéresis
************************************************************************
************/
void SiNo::ConfigurarHisteresis(float PRENDE, float APAGA)
{
  Prende = PRENDE;
  Apaga = APAGA;
  Pendiente = 1;          // Normalmente, el actuador hace que suba la
ENTRADA = MEDICION
                          // (medición de la variable controlada).
  if (Apaga<Prende) Pendiente = -1;
  // Esto puede ocurrir cuando se toma como ENTRADA a la señal ERROR
  // (ERROR = REFERENCIA - MEDICION).
  // También cuando, por ejemplo, se controla la potencia de
  // una heladera, que hace que baje la temperatura.
}

//**********************************************************************
*************
```

velocidad.h

```
/*****************************************************************************
***
* Velocidad de cambios de estado
*****************************************************************************
***
* Archivo: velocidad.h
* Resumen: El módulo reúne algunas funciones para calcular la velocidad
*          con que un pin cambia de estado.
* Autor:   Guillermo Caporaletti <gfcaporaletti@undav.edu.ar>
* Fecha:   Abril de 2023
*****************************************************************************
**/

/*****************************************************************************
***
* Libreria y módulos incluidas
*****************************************************************************
**/

#include "Arduino.h"

/*****************************************************************************
***
* Constantes definidas
*****************************************************************************
**/

#define MAX_CAMBIOS_RPM       300                   // Máxima cantidad de
cambios de estado por minuto, físicamente posible
                                                    // (se recomienda poner
un 50% más que el real).
#define MAX_CAMBIOS_PS        MAX_CAMBIOS_RPM/60    // Máxima cantidad de
cambios de estado por segundo, físicamente posible.
#define CAMBIOS_P_VUELTA      20                    // Cambios de estado
producidos en cada vuelta.
#define MIN_US_ENTRE_CAMBIOS  1000000 / ( MAX_CAMBIOS_PS * CAMBIOS_P_VUELTA
)
                                                    // Máximo tiempo posible
entre cambios de estado.
#define CTE_VEL               60 * 1000000 / CAMBIOS_P_VUELTA
                                                    // Constante de
velocidad (cambios por vuelta en micro)

/*****************************************************************************
***
* Funciones públicas
*****************************************************************************
```

```
**/

void ConfigurarVelocidad(byte PIN);                 // Configura el pin a leer
e inicializa variables
int velocidadRPM();                                 // Mide la velocidad en
función de las interrupciones que haya habido

/*************************************************************************
***
* FIN DE ARCHIVO velocidad.h
*************************************************************************
**/
```

velocidad.cpp

```
/*****************************************************************************
***
* Velocidad de cambios de estado
*****************************************************************************
***
* Archivo: velocidad.cpp
* Resumen: El módulo reúne algunas funciones para calcular la velocidad
*          con que un pin cambia de estado.
* Autor:   Guillermo Caporaletti <gfcaporaletti@undav.edu.ar>
* Fecha:   Abril de 2023
*****************************************************************************
**/

/*****************************************************************************
***
* Libreria y módulos incluidas
*****************************************************************************
**/

#include "velocidad.h"

/*****************************************************************************
***
* Variables privadas (para manejo de la ISR)
*****************************************************************************
**/

byte PinEntrada;                      // Pin de entrada cuya velocidad de
cambio de estado deseamos medir
volatile unsigned long usAnterior=0;  // Almacena el último tiempo de la
interrupción en microsegundos.
volatile unsigned long usDelta=0;     // Almacena cuánto pasó entre cada
interrupción en microsegundos.
volatile unsigned long contadorISR=0; // Cuenta cambios de estado: otro modo
de registro.
volatile bool estadoSensor;           // Guarda estado último para evitar
falsas detecciones de cambios de estado.
int velocidadAnterior = 0;            // Esto puede usarse para promediar
las velocidades medidas

/*****************************************************************************
***
* Funciones privadas
*****************************************************************************
**/

// ISR que calcula cuánto pasó desde la anterior interrupción
```

```
// La respuesta en velocidad la da otra función (que es pública)
void ISRcalculaDelta()
{
  static bool tilt = true;
  if (digitalRead(PinEntrada) != estadoSensor && micros() - usAnterior >
MIN_US_ENTRE_CAMBIOS)
  {
    estadoSensor=!estadoSensor;       // Cambió el estado del sensor
    usDelta=(micros()-usAnterior);    // Calcula delta (sin filtro
promediador)
    contadorISR++;                    // Agrega cuenta
    usAnterior=micros();           // Actualiza
    // usDelta --> Almacena cuanto pasó entre cada interrupción en
microsegundos.
    // usAnterior --> Almacena el último tiempo de la interrupción en
microsegundos.


  }
}

/**************************************************************************
***
* Funciones privadas
**************************************************************************
**/

// Configura la interrupción
void ConfigurarVelocidad(byte PIN)
{
  PinEntrada = PIN;             // Almacenamos el pin que debo leer
  pinMode(PinEntrada, INPUT);   // Lo seteo como entrada digital
  attachInterrupt(digitalPinToInterrupt(PinEntrada),ISRcalculaDelta,RISING);
              // Configuro la función a llamar en cada interrupción
  estadoSensor = digitalRead(PinEntrada);
              // Muestro el estado inicial!!!
              // Verifico incialmente en qué estado está el sensor y lo
guardo en la variable.
}

// Devuelve la velocidad en RPM según el tiempo entre cambios de estado
int velocidadRPM()
{
  int med = 0;
  if (usAnterior==0) {
    // Aún no hubo ninguna interrupción
    // Supongo una velocidad de 0 RPM
    med = 0;
```

```
  } else if ((micros()-usAnterior)>usDelta) {
    // micros()--> es el tiempo que paso desde que se inicio el programa en
microsegundos
    // usAnterior --> el delta de la ultima interrupcion
    // usDelta --> el tiempo q paso entre la interrupcion anterior y la
actual
    // Quiere decir que la velocidad real es menor a la que indica msDelta
    // (Esto puede ocurrir porque msDelta sólo se actualiza cuando ocurre la
interrupción)
    med = (CTE_VEL/(micros()-usAnterior));
  } else {
    // Calculo con msDelta :-)
    med = (CTE_VEL/usDelta);
    //3000RPM * 4 VUELTAS = 12000
    //12000/60= 200
    //1/200 = 0.005SEG
  }

  // Ya tengo un valor med:
  // Puedo limitar y/o promediar su valor.
  // med = min(med, 1.2*MAX_CAMBIOS_RPM);
  // med = 0.7*velocidadAnterior + 0.3*med;
  // velocidadAnterior = med;

  // Listo!!!
  return med;
}

/*****************************************************************************
***
* FIN DE ARCHIVO velocidad.cpp
*****************************************************************************
**/
```

Anexo 2: Medición 1

A continuación se coloca el código fuente elaborado para el control automático de la cinta transportadora, que fue usado durante la primera medición.

TLMed1.ino

```
/*****************************************************************************
*************
* CONTROL SI-NO
*****************************************************************************
*************
* Ejemplo para la libreria controlSiNo.h
* Leemos una entrada analógica y mostramos en LED_BUILTIN y en el monitor
* el resultado de la función de contro si-no.
*
* UNDAV, Sistemas de Control Automático, 2022.
*****************************************************************************
************/

/*****************************************************************************
*************
* Libreria incluídas
*****************************************************************************
************/
#include "controlSiNo.h"
#include "velocidad.h"

/*****************************************************************************
*************
* Constantes
*****************************************************************************
************/
const byte pinVelocidad = 21; // En este pin tenemos el sensor optoacoplador
const byte pinLedEmergencia = 13;  // Pin del led que trae integrada la
placa arduino
const byte pinMotor = 5; // Pin que, a traves de un rele, ira al motor para
encenderlo o apagarlo

const byte pinBoton = 30;
const long tiempoDelta = 10; // Intervalo de tiempo entre muestras (en
milisegundos). Medicion: 160 cambios de estado por segundo -- 160Hz = 6,25ms

const unsigned int Objetivo = 150; // Como prueba, elegimos un objetivo de
150RPM
const int Histeresis = 20; // Como prueba, elegimos que la histeresis varie
entre 130RPM y 170RPM
```

```
//Constantes de PWM
const int MinPWM = 0;
const int MaxPWM = 255;

/*****************************************************************************
*************
* Variables globales
*****************************************************************************
************/
//int objetivoRPM = 0;
int cintaRPM = 0;

SiNo indicadorMotor(Objetivo-Histeresis, Objetivo+Histeresis);
bool estadoIndicador = false;

unsigned long tiempoAnterior = 0;
unsigned long tiempoInicial = 0;

/*****************************************************************************
*************
* Funciones
*****************************************************************************
************/
void setup() {
  // Seteos iniciales
  ConfigurarVelocidad(pinVelocidad);
  pinMode(pinBoton, INPUT);
  pinMode(pinLedEmergencia, OUTPUT);
  pinMode(pinMotor, OUTPUT);
  Serial.begin(9600);

  digitalWrite(pinLedEmergencia, LOW); // Apago el led.

  // Primer valor de la serie
  tiempoAnterior = millis();
  tiempoInicial = tiempoAnterior;
  estadoIndicador = indicadorMotor.Estado();  // Tomo estado inicial
  Serial.println(MIN_US_ENTRE_CAMBIOS);

  // Escribo encabezado
  Serial.println("Tiempo\tMedRPM\tEstado");
  Serial.println("[ms]\t[1024]\t[0;1]");
  mostrarEstadisticas();                // Muestro el estado inicial


}

/*****************************************************************************
```

```
**********/
void loop() {
  // Si se apreta el boton de emergencia, se debe parar completamente el
funcionamiento
  if (digitalRead(pinBoton) == HIGH) paramosSistema();

  if(millis() >= (tiempoAnterior + tiempoDelta)){ //cda tiempoDelta
    tiempoAnterior += tiempoDelta;  // Actualizo el tiempo

    // Leo
    realizarMediciones();

    // Controlo
    estadoIndicador = indicadorMotor.Controlar(cintaRPM);

    if(estadoIndicador){
      digitalWrite(pinMotor, LOW);
    }
    else digitalWrite(pinMotor, HIGH);

    // Muestro
    mostrarEstadisticas();
  }

}

/**********************************************************************
**********/
void realizarMediciones(){
  //objetivoRPM = lecturaRPM(pin); // Aqui leemos la velocidad de la
referencia; como no usamos un potenciometro de referencia, deberiamos usar
solamente Objetivo = 180.
  cintaRPM = velocidadRPM(); // Leemos la velocidad de la cinta
}

/*****************************************Esta función no se
utiliza*****************************************/
int lecturaRPM(byte pin)
{
  // Lectura de la entrada;
  int valorEntrada = analogRead(pin);
  // Mapeo de valor entre 0 y maxPRM
  int objetivoRPM = map(valorEntrada, 0, 1023, 0, MAX_CAMBIOS_RPM);

  //roundup to 50
  //objetivoRPM = round((float) objetivoRPM/50.0)*50;
  objetivoRPM = (( objetivoRPM+50) /250)*250;
```

```
  // sacar promedio de potenciometro roundup 50/100
  if (objetivoRPM < 150) {
    objetivoRPM = 0;
  }
  return objetivoRPM;
}

/*****************************************************************************
************/
void paramosSistema(void)
{
   digitalWrite(pinMotor, HIGH);

   Serial.println("Se acciono el boton de emergencia");
   Serial.println("El motor se esta deteniendo");

   while (1) {
    digitalWrite(pinLedEmergencia, LOW);
    delay(1000);
    digitalWrite(pinLedEmergencia, HIGH);
    delay(500);
   } //Para salir del estado de parada de emergencia, se debe reiniciar el
sistema
}

/*****************************************************************************
************/
void mostrarEstadisticas() {
  Serial.print(tiempoAnterior - tiempoInicial);
  Serial.print("\t");
  Serial.print(cintaRPM);
  Serial.print("\t");
  Serial.println(estadoIndicador);
}
```

Anexo 3: Medición 2

A continuación se coloca el código fuente elaborado para el control automático de la cinta transportadora, que fue usado durante la segunda medición.

TLMed2.ino

```
/*****************************************************************************
*************
* CONTROL SI-NO
*****************************************************************************
*************
* Ejemplo para la librería controlSiNo.h
* Leemos una entrada analógica y mostramos en LED_BUILTIN y en el monitor
* el resultado de la función de control si-no.
*
* UNDAV, Sistemas de Control Automático, 2022.
*****************************************************************************
************/

/*****************************************************************************
*************
* Libreria incluídas
*****************************************************************************
************/
#include "controlSiNo.h"
#include "velocidad.h"

/*****************************************************************************
*************
* Constantes
*****************************************************************************
************/
const byte pinVelocidad = 21; // En este pin tenemos el sensor optoacoplador
const byte pinLedEmergencia = 13;  // Pin del led que trae integrada la
placa arduino
const byte pinMotor = 5; // Pin que, a través de un rele, ira al motor para
encenderlo o apagarlo

const byte pinBoton = 30;
const long tiempoDelta = 5; // Intervalo de tiempo entre muestras (en
milisegundos). Medicion: 160 cambios de estado por segundo -- 160Hz = 6,25ms

const unsigned int Objetivo = 100; // Como prueba, elegimos un objetivo de
100RPM
const int Histeresis = 80; // Como prueba, elegimos que la histeresis varie
entre 40RPM y 180RPM
```

```
//Constantes de PWM
const int MinPWM = 0;
const int MaxPWM = 255;

/*********************************************************************************
*************
* Variables globales
*********************************************************************************
************/
//int objetivoRPM = 0;
int cintaRPM = 0;

SiNo indicadorMotor(Objetivo-Histeresis, Objetivo+Histeresis);
bool estadoIndicador = false;

unsigned long tiempoAnterior = 0;
unsigned long tiempoInicial = 0;

/*********************************************************************************
*************
* Funciones
*********************************************************************************
************/
void setup() {
  // Seteos iniciales
  ConfigurarVelocidad(pinVelocidad);
  pinMode(pinBoton, INPUT);
  pinMode(pinLedEmergencia, OUTPUT);
  pinMode(pinMotor, OUTPUT);
  Serial.begin(9600);

  digitalWrite(pinLedEmergencia, LOW); // Apago el led.

  // Primer valor de la serie
  tiempoAnterior = millis();
  tiempoInicial = tiempoAnterior;
  estadoIndicador = indicadorMotor.Estado();  // Tomo estado inicial
  Serial.println(MIN_US_ENTRE_CAMBIOS);

  // Escribo encabezado
  Serial.println("Tiempo\tMedRPM\tEstado");
  Serial.println("[ms]\t[1024]\t[0;1]");
  mostrarEstadisticas();                // Muestro el estado inicial


}
```

```
/*************************************************************************
***********/
void loop() {
  // Si se apreta el boton de emergencia, se debe parar completamente el
funcionamiento
  if (digitalRead(pinBoton) == HIGH) paramosSistema();

  if(millis() >= (tiempoAnterior + tiempoDelta)){ //cda tiempoDelta
    tiempoAnterior += tiempoDelta;  // Actualizo el tiempo

    // Leo
    realizarMediciones();

    // Controlo
    estadoIndicador = indicadorMotor.Controlar(cintaRPM);

    if(estadoIndicador){
      digitalWrite(pinMotor, LOW);
    }
    else digitalWrite(pinMotor, HIGH);

    // Muestro
    mostrarEstadisticas();
  }

}

/*************************************************************************
***********/
void realizarMediciones(){
  //objetivoRPM = lecturaRPM(pin); // Aqui leemos la velocidad de la
referencia; como no usamos un potenciometro de referencia, deberiamos usar
solamente Objetivo = 180.
  cintaRPM = velocidadRPM(); // Leemos la velocidad de la cinta
}

/*****************************************Esta función no se
utiliza*********************************************/
int lecturaRPM(byte pin)
{
  // Lectura de la entrada;
  int valorEntrada = analogRead(pin);
  // Mapeo de valor entre 0 y maxPRM
  int objetivoRPM = map(valorEntrada, 0, 1023, 0, MAX_CAMBIOS_RPM);

  //roundup to 50
  //objetivoRPM = round((float) objetivoRPM/50.0)*50;
  objetivoRPM = (( objetivoRPM+50) /250)*250;
```

```
  // sacar promedio de potenciometro roundup 50/100
  if (objetivoRPM < 150) {
    objetivoRPM = 0;
  }
  return objetivoRPM;
}

void aumentarVelocidad(int cantidad){
  velocidad += cantidad;
  if(velocidad > MaxPWM) velocidad = MaxPWM;
  else if (velocidad < MinPWM) velocidad = MinPWM;
  motor.setSpeed(velocidad);
}

/*****************************************************************************
************/
void paramosSistema(void)
{
   digitalWrite(pinMotor, HIGH);

   Serial.println("Se acciono el boton de emergencia");
   Serial.println("El motor se esta deteniendo");

   while (1) {
    digitalWrite(pinLedEmergencia, LOW);
    delay(1000);
    digitalWrite(pinLedEmergencia, HIGH);
    delay(500);
   } //Para salir del estado de parada de emergencia, se debe reiniciar el
sistema
}

/*****************************************************************************
************/
void mostrarEstadisticas() {
  Serial.print(tiempoAnterior - tiempoInicial);
  Serial.print("\t");
  Serial.print(cintaRPM);
  Serial.print("\t");
  Serial.println(estadoIndicador);
}
```

Anexo 4: Medición 3

A continuación se coloca el código fuente elaborado para el control automático de la cinta transportadora, que fue usado durante la tercera medición, en la cual simulamos un peso.

TLMed3.ino

```
/*****************************************************************************
*************
* CONTROL SI-NO
*****************************************************************************
*************
* Ejemplo para la libreria controlSiNo.h
* Leemos una entrada analógica y mostramos en LED_BUILTIN y en el monitor
* el resultado de la función de contro si-no.
*
* UNDAV, Sistemas de Control Automático, 2022.
*****************************************************************************
************/

/*****************************************************************************
*************
* Libreria incluídas
*****************************************************************************
************/
#include "controlSiNo.h"
#include "velocidad.h"

/*****************************************************************************
*************
* Constantes
*****************************************************************************
************/
const byte pinVelocidad = 21; // En este pin tenemos el sensor optoacoplador
const byte pinLedEmergencia = 13;  // Pin del led que trae integrada la
placa arduino
const byte pinMotor = 5; // Pin que, a traves de un rele, ira al motor para
encenderlo o apagarlo

const byte pinBoton = 30;
const long tiempoDelta = 5; // Intervalo de tiempo entre muestras (en
milisegundos). Medicion: 160 cambios de estado por segundo -- 160Hz = 6,25ms

const unsigned int Objetivo = 130; // Como prueba, elegimos un objetivo de
130RPM, teniendo en cuenta que simularemos una carga
```

```
const int Histeresis = 20; // Como prueba, elegimos que la histeresis varie
entre 110RPM y 150RPM

//Constantes de PWM
const int MinPWM = 0;
const int MaxPWM = 255;

/*************************************************************************
*************
* Variables globales
*************************************************************************
************/
//int objetivoRPM = 0;
int cintaRPM = 0;

SiNo indicadorMotor(Objetivo-Histeresis, Objetivo+Histeresis);
bool estadoIndicador = false;

unsigned long tiempoAnterior = 0;
unsigned long tiempoInicial = 0;

/*************************************************************************
*************
* Funciones
*************************************************************************
************/
void setup() {
  // Seteos iniciales
  ConfigurarVelocidad(pinVelocidad);
  pinMode(pinBoton, INPUT);
  pinMode(pinLedEmergencia, OUTPUT);
  pinMode(pinMotor, OUTPUT);
  Serial.begin(9600);

  digitalWrite(pinLedEmergencia, LOW); // Apago el led.

  // Primer valor de la serie
  tiempoAnterior = millis();
  tiempoInicial = tiempoAnterior;
  estadoIndicador = indicadorMotor.Estado();  // Tomo estado inicial
  Serial.println(MIN_US_ENTRE_CAMBIOS);

  // Escribo encabezado
  Serial.println("Tiempo\tMedRPM\tEstado");
  Serial.println("[ms]\t[1024]\t[0;1]");
  mostrarEstadisticas();                // Muestro el estado inicial
```

```
}

/*************************************************************************
************/
void loop() {
  // Si se apreta el boton de emergencia, se debe parar completamente el
funcionamiento
  if (digitalRead(pinBoton) == HIGH) paramosSistema();

  if(millis() >= (tiempoAnterior + tiempoDelta)){ //cda tiempoDelta
    tiempoAnterior += tiempoDelta;  // Actualizo el tiempo

    // Leo
    realizarMediciones();

    // Controlo
    estadoIndicador = indicadorMotor.Controlar(cintaRPM);

    if(estadoIndicador){
      digitalWrite(pinMotor, LOW);
    }
    else digitalWrite(pinMotor, HIGH);

    // Muestro
    mostrarEstadisticas();
  }

}

/*************************************************************************
************/
void realizarMediciones(){
  //objetivoRPM = lecturaRPM(pin); // Aqui leemos la velocidad de la
referencia; como no usamos un potenciometro de referencia, deberiamos usar
solamente Objetivo = 180.
  cintaRPM = velocidadRPM(); // Leemos la velocidad de la cinta
}

/*****************************************Esta función no se
utiliza*****************************************/
int lecturaRPM(byte pin)
{
  // Lectura de la entrada;
  int valorEntrada = analogRead(pin);
  // Mapeo de valor entre 0 y maxPRM
  int objetivoRPM = map(valorEntrada, 0, 1023, 0, MAX_CAMBIOS_RPM);

  //roundup to 50
```

```
  //objetivoRPM = round((float) objetivoRPM/50.0)*50;
  objetivoRPM = (( objetivoRPM+50) /250)*250;

  // sacar promedio de potenciometro roundup 50/100
  if (objetivoRPM < 150) {
    objetivoRPM = 0;
  }
  return objetivoRPM;
}

void aumentarVelocidad(int cantidad){
  velocidad += cantidad;
  if(velocidad > MaxPWM) velocidad = MaxPWM;
  else if (velocidad < MinPWM) velocidad = MinPWM;
  motor.setSpeed(velocidad);
}

/*****************************************************************************
************/
void paramosSistema(void)
{
   digitalWrite(pinMotor, HIGH);

   Serial.println("Se acciono el boton de emergencia");
   Serial.println("El motor se esta deteniendo");

   while (1) {
    digitalWrite(pinLedEmergencia, LOW);
    delay(1000);
    digitalWrite(pinLedEmergencia, HIGH);
    delay(500);
   } //Para salir del estado de parada de emergencia, se debe reiniciar el
sistema
}

/*****************************************************************************
************/
void mostrarEstadisticas() {
  Serial.print(tiempoAnterior - tiempoInicial);
  Serial.print("\t");
  Serial.print(cintaRPM);
  Serial.print("\t");
  Serial.println(estadoIndicador);
}
```

Bibliografía

Altronics (Sin Fecha). *Sensor de Velocidad infrarrojo FC-03.* Altronics. Recuperado el 20/04/2023 de: https://altronics.cl/sensor-velocidad-ir-fc03

Campbell, S. (2015). *How to set up a 5v relay on the Arduino.* Circuit Basics. Recuperado el 21/04/2023 de: https://www.circuitbasics.com/setting-up-a-5v-relay-on-the-arduino/

Del Valle, L. (2019). *Resistencia pull up y pull down.* Programar Facil. Recuperado el 20/04/2023 de: https://programarfacil.com/blog/arduino-blog/resistencia-pull-up-y-pull-down/

Hernández Gaviño, R. (2010). *Introducción a los sistemas de control: Conceptos, aplicaciones y simulación con MATLAB. Primera edición.* México: Pearson Educación.

Joe (2020, 21 de octubre). *El principio de Funcionamiento Del Acoplador Óptico.* Omch. Recuperado el 21/04/2023 de: https://www.omch.co/es/the-working-principle-of-optical-coupler/

Kuo, B. C. (1996). *Sistemas de Control Automático. Séptima Edición.* México: Prentice Hall Hispanoamericana.

Robótica Fácil (Sin Fecha). *Rele Doble 5V 10A 250Vac.* Robótica Fácil. Recuperado el 20/04/2023 de: https://roboticafacil.es/prod/rele-doble-5v-10a-250vac/

Sritu Hobby (2021). *IR infrared speed sensor with Arduino | How does work IR speed sensor.* SrituHobby. Recuperado el 27/04/2023 de: https://srituhobby.com/ir-infrared-speed-sensor-with-arduino-how-does-work-ir-speed-sensor/

Wikipedia (2020, 3 de febrero). *Sistema Invariante en el Tiempo.* Wikipedia. Recuperado el 18/04/2023 de: https://es.wikipedia.org/wiki/Sistema_invariante_en_el_tiempo

Referencias

Figura 2: *Especificaciones técnicas de Arduino Mega 2560*. Extraído el 18 de abril del 2023 de https://store-usa.arduino.cc/products/arduino-mega-2560-rev3

Figura 3*: Diagrama de pines del Arduino Mega 2560 Rev3.* Extraído el 18 de abril del 2023 de https://store-usa.arduino.cc/products/arduino-mega-2560-rev3

Figura 4: *Terminales y Pines de un Relay de 5V.* Extraído el 18 de abril del 2023 de https://chitoraspberrypi.blogspot.com/2017/05/el-rele-algo-de-teoria.html

Figura 5: *Modelo de 2 Relés utilizado en la práctica.* Extraído el 18 de abril del 2023 de https://laelectronica.com.gt/modulo-rele-5v-2-canales

Figura 6: *Plaqueta FC-03 - LM393.* Extraído el 19 de abril del 2023 de https://french.alibaba.com/p-detail/Photoelectric-1600760025154.html

Figura 7: *Diagrama del Optoacoplador.* Extraído el 19 de abril del 2023 de https://srituhobby.com/ir-infrared-speed-sensor-with-arduino-h

Figura 8: *Botón pulsador grande.* Extraído el 19 de abril del 2023 de https://compras.patagoniatec.com/productos/boton-pulsador-grande-normal-abierto/

Figura 9: *Configuración de resistencia en Pull Down o Pull Up*. Extraído el 19 de abril del 2023 de https://programarfacil.com/blog/arduino-blog/resistencia-pull-up-y-pull-down/